Kubernetes: Pods, ReplicaSets & Cluster Networking

How the Kubernetes control plane orchestrates atomic containers, manages self-healing ReplicaSets, and routes flat cluster overlay networks.

1. The Anatomy of a Pod

A Pod is the atomic unit in Kubernetes. Containers within a Pod share the same network namespace (via `pause` container) and talk over `localhost`.

```
Pod [ pause_container (eth0: 10.244.1.15) ]
├─ app_container (port 8080)
└─ sidecar_proxy (port 9090) ➔ localhost:8080
```

2. ReplicaSets & Self-Healing Loop

Kube-Controller-Manager runs an infinite reconciliation loop: if a node crashes and active Pod count drops below `replicas: 3`, a new Pod is scheduled immediately.

```
Desired State (spec) == Actual State (status)
```

3. Cluster CNI Flat Overlay Network

Every Pod gets a unique, real routable IP across the entire cluster without NAT (using Flannel, Calico, or Cilium eBPF):

```
Node 1 (Pod 10.244.1.5) ➔ VXLAN Geneve Overlay ➔ Node 2 (Pod 10.244.2.8)
```

4. Production Kubectl Commands

Essential CLI workflows for cluster debugging:

```
# Inspect Pod IPs and scheduled nodes
kubectl get pods -o wide --all-namespaces

# Scale ReplicaSet dynamically
kubectl scale rs my-api --replicas=5
```